



C3P

Projet 2 : Démineur

Olivia LANG
Julien MIROUX
Phuong Lan NGUYEN

[github](#)

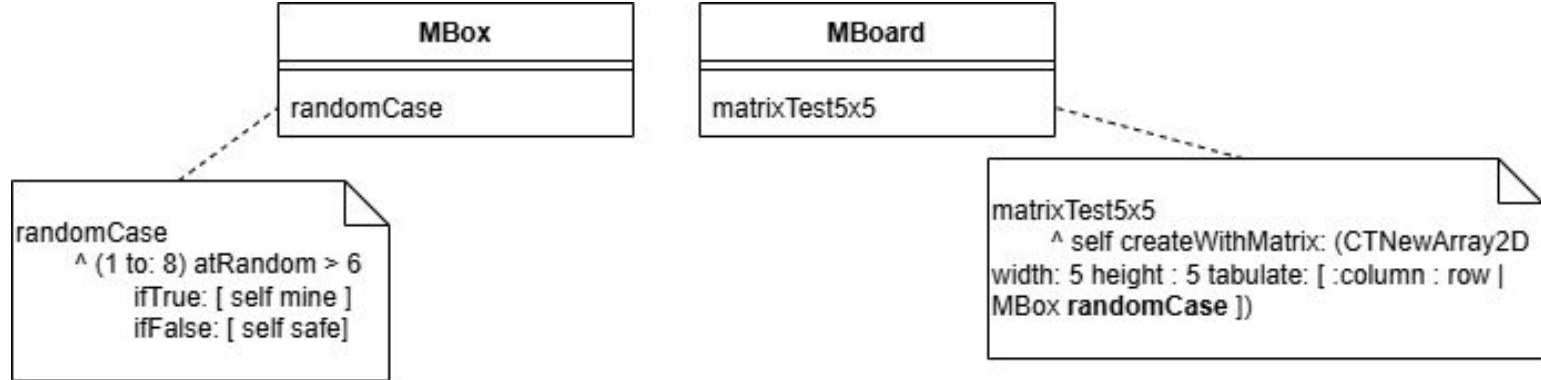
Master 1 MIAGE - 11/12/2025

Sommaire



- I. Introduire de nouveaux algorithmes pour placer les mines
- II. Introduire la possibilité de modifier la taille du plateau
- III. Compter le nombre de sélection et afficher le nombre de mines

Design initial



> MBox s'occupe de se définir en tant que mine ou safe : 2 chances sur 8 d'être une mine !

Nouvelles fonctionnalités

Introduire de nouveaux algorithmes pour placer les mines :

une mine sur chaque
colonne du plateau

0	0	1	2	🔧
0	0	1	🔧	2
0	1	2	2	1
2	3	🔧	1	0
🔧	🔧	2	1	0

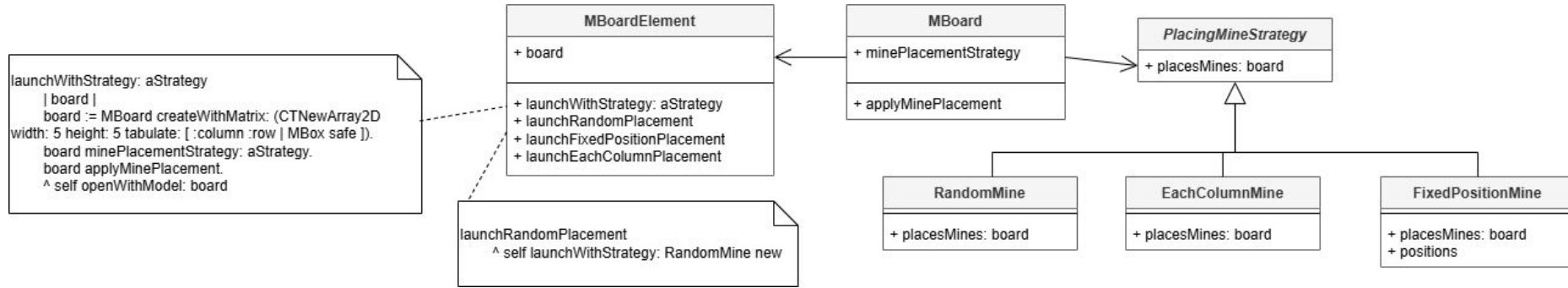
des mines à des positions fixées

0	1	1	1	0
1	2	🔧	1	0
🔧	3	3	2	1
2	🔧	2	🔧	1
1	1	2	1	1

une mine aléatoirement placée

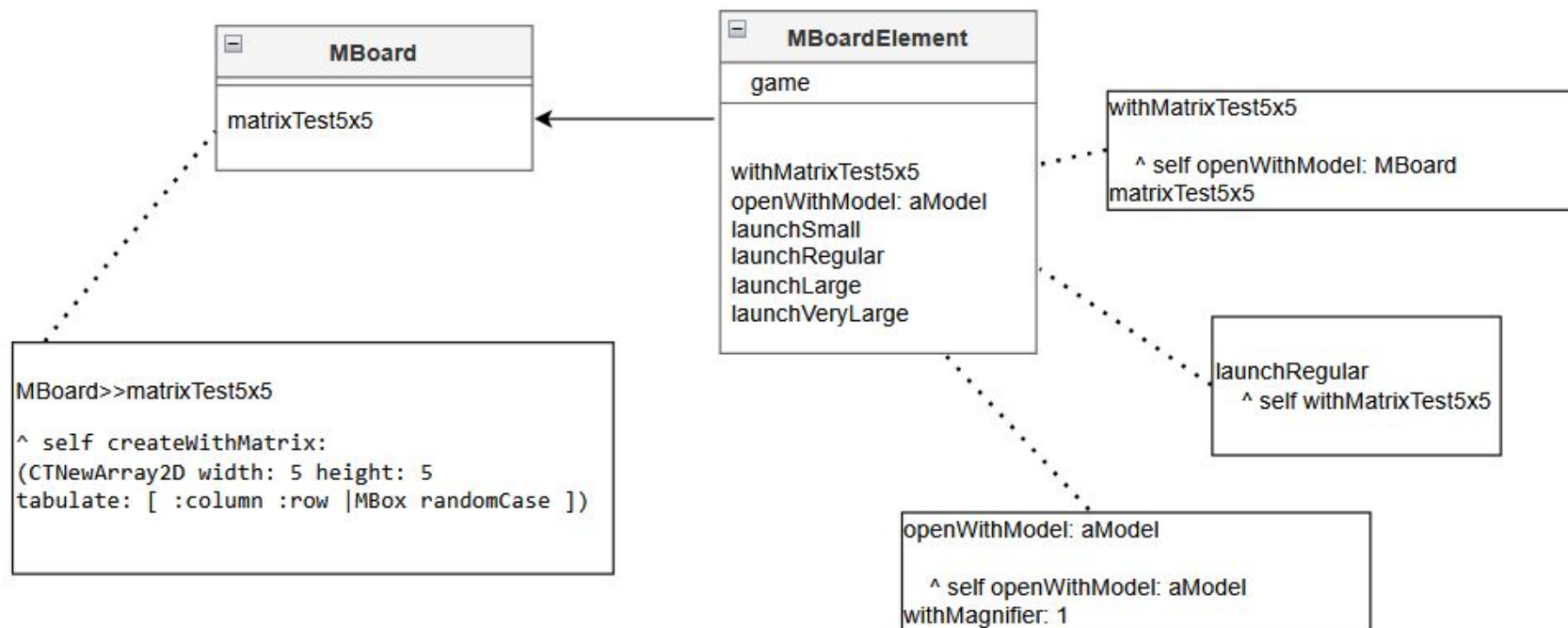
🔧	2	🔧	1	0
2	3	2	3	2
🔧	4	3	🔧	🔧
🔧	🔧	🔧	4	3
🔧	🔧	3	2	🔧

Nouveaux designs



Strategy	Sans design pattern
⊕ éviter les bloc de conditions ⊕ ne pas modifier le comportement de MBoard ⊕ ajouter des fonctionnalités sans modifier le code existant ⊖ gestion de beaucoup de classes (1 stratégie = 1 classe)	⊖ modification dans la classe MBoard ou MBox (avec des blocs de condition) ⊖ difficile à maintenir (ajout de n algorithmes) ⊖ pas de séparation des responsabilités ⊕ moins de classes ⊕ plus simple à mettre en place

Design initial



Nouvelles fonctionnalités

Introduire la possibilité d'avoir un plateau de jeu de différentes tailles.
Par exemple 5X5, 10X10, 30x20...

1	⚙	⚙	⚙	⚙
2	5	⚙	6	3
⚙	4	⚙	5	⚙
2	4	⚙	4	⚙
⚙	2	1	2	1

Board : 5X5

0	1	1	1	0	0	0	0
1	2	⚙	1	0	0	0	0
⚙	3	3	2	1	0	0	0
2	⚙	2	⚙	1	0	0	0
1	1	2	1	1	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

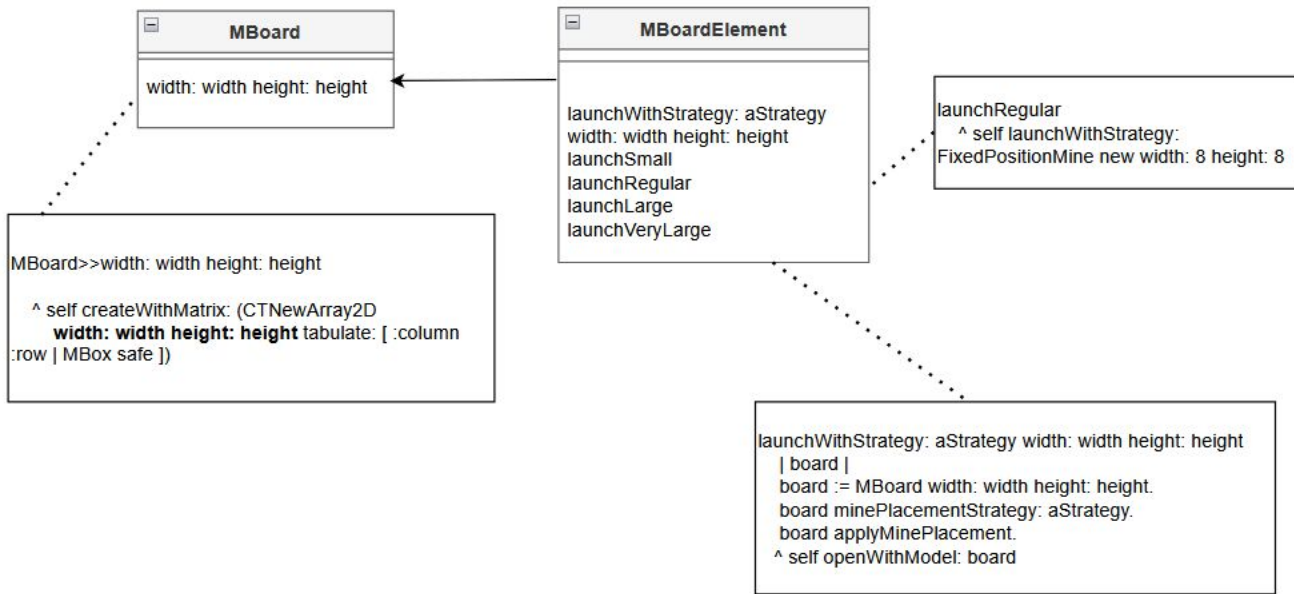
Board : 8X8

⚙	⚙	⚙	⚙	⚙	3	1	1	⚙	2	⚙	1
⚙	5	⚙	⚙	⚙	⚙	2	3	2	3	2	2
1	3	5	⚙	6	5	⚙	4	⚙	3	3	⚙
2	3	⚙	⚙	⚙	3	⚙	5	⚙	4	⚙	⚙
⚙	⚙	3	3	3	3	3	⚙	4	5	⚙	⚙
⚙	4	3	2	3	⚙	3	3	⚙	⚙	3	2
2	⚙	3	⚙	⚙	5	⚙	4	5	4	3	1
3	3	4	⚙	⚙	⚙	4	⚙	⚙	⚙	5	⚙
⚙	⚙	4	5	⚙	5	5	⚙	6	⚙	⚙	⚙
⚙	5	⚙	⚙	6	⚙	⚙	3	4	⚙	⚙	3
⚙	4	5	⚙	⚙	⚙	5	⚙	4	3	4	2
2	⚙	3	⚙	4	2	3	⚙	3	⚙	2	⚙

Board : 12X12

Nouveau design

Factory Method 



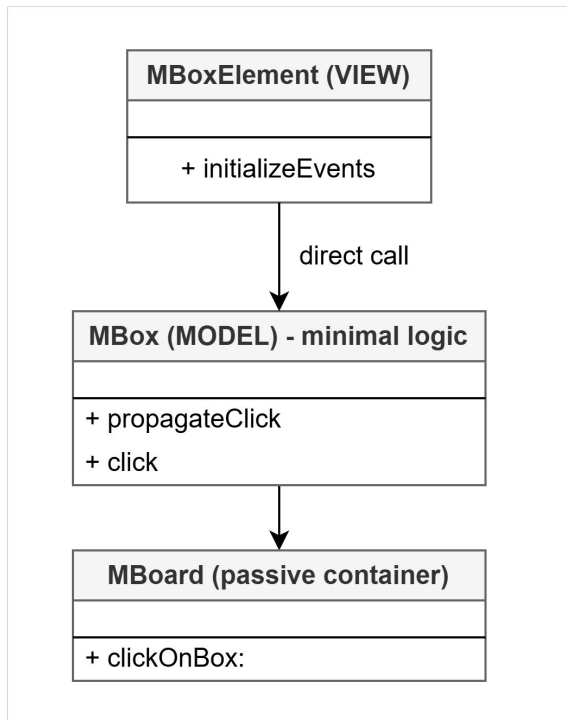
Factory Method

- ⊕ Flexibilité (Dynamique)
- ⊕ Facile à maintenir
- ⊕ Lisibilité
- ⊕ Encapsulation
- ⊖ Problème de l'explosion des paramètres

Sans design pattern | Héritage

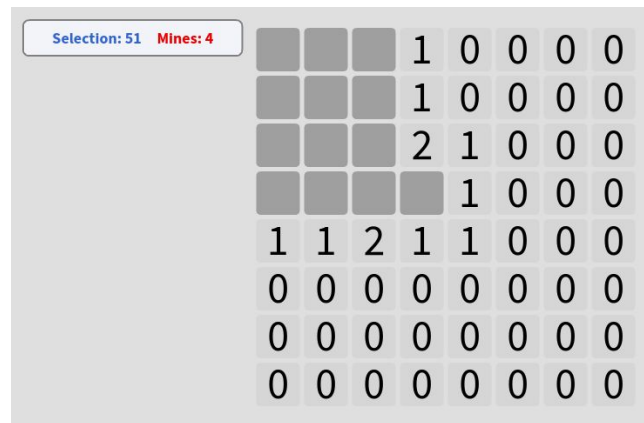
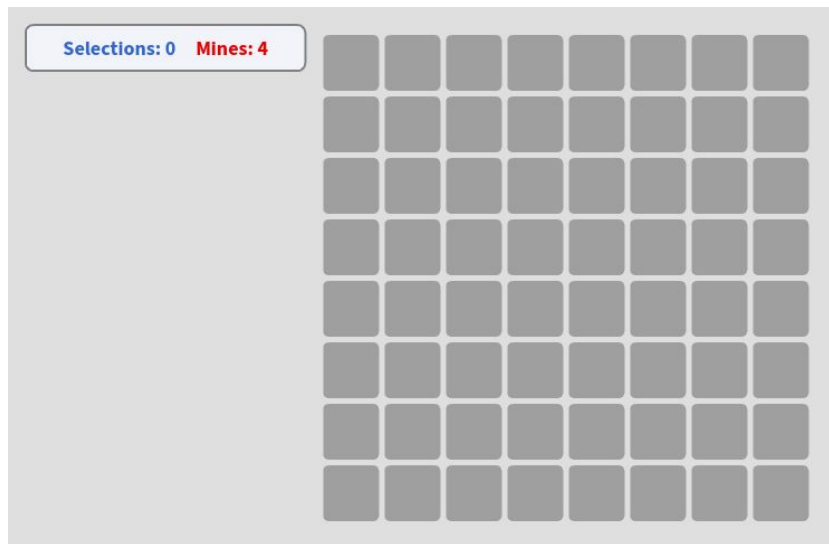
- ⊖ Explosion de classes pour chaque taille de tableau / Rigidité (Static)
- ⊖ Duplication de code
- ⊕ Facile à exécuter

Design initial

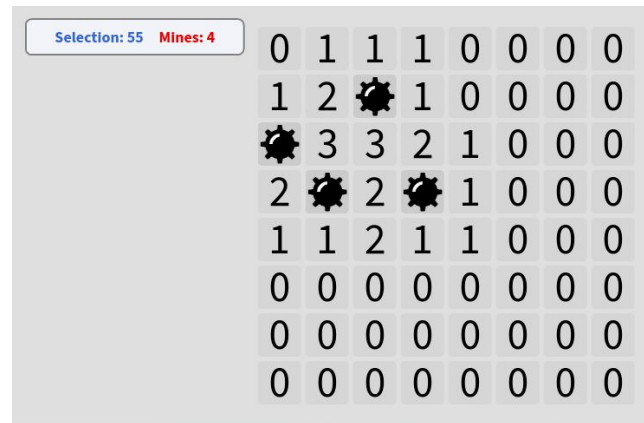


Nouvelles fonctionnalités

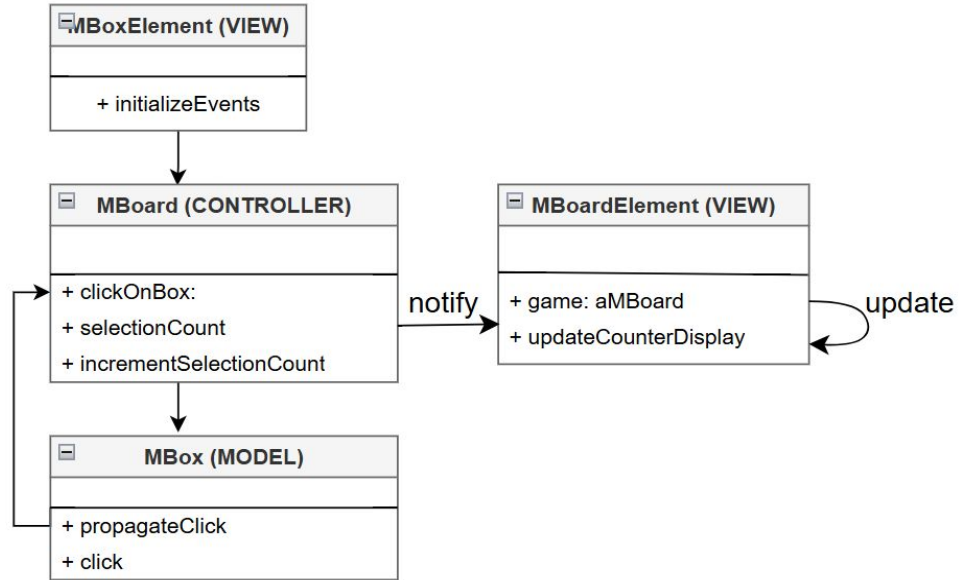
Initialize du board



Après click



Nouveaux designs



Observe design pattern + MVC architecture

- ⊕ Avec état et validation
- ⊕ Couplage faible : l'UI appelle uniquement Board => Respect MVC
- ⊕ `propagateClick` a déjà compté les clics.
- ⊖ Plus complexe

Sans design pattern | MVC architecture

- ⊖ Pas d'état, pas de validation
- ⊖ Couplage fort : l'UI connaît trop le modèle
- ⊖ `propagateClick` ne compte pas les clics.
- ⊕ Facile à exécuter